



UNIVERSIDAD AUTÓNOMA DE SAN LUIS POTOSÍ
FACULTAD DE ESTUDIOS PROFESIONALES ZONA MEDIA

Práctica 13: Control de LED RGB mediante señales PWM

Nombre del alumno: Sergio Adolfo Juárez Mendoza

Clave: 369376

Profesor: Ing. Jesús Padrón

Fecha de entrega: 21 de abril de 2025

Introducción

El objetivo de esta práctica es familiarizarse con el uso de señales PWM (modulación por ancho de pulso) como método de control en dispositivos como LEDs RGB. Esta técnica permite variar la intensidad de cada canal de color (rojo, verde y azul) para generar una gama diversa de colore

Objetivo

poder jugar con las luces de un led RGB para asi poder entender mas su funcionamiento y asi poder familiarizarnos con el mismo

1 Material y Recursos Necesarios

1.1 Hardware

- Raspberry Pi Pico W
- LED RGB
- 3 potenciómetros de 10 K
- 3 resistencias de 300
- Protoboard
- Cables de conexión calibre 22 AWG

1.2 Software y Librerías

- Visual Studio Code con soporte para Raspberry Pi Pico
- Librerías:
 - `pico/stdlib.h`
 - `hardware/adc.h`

Desarrollo

creacion del codigo

```
1 #include <stdio.h>
2 #include "pico/stdlib.h"
3 #include "hardware/adc.h"
4 #include "hardware/pwm.h"
5
6 #define PIN_ROJO 16
7 #define PIN_VERDE 18
8 #define PIN_AZUL 17
9
10 void init_pwm(uint pin) {
11     gpio_set_function(pin, GPIO_FUNC_PWM);
12     uint slice = pwm_gpio_to_slice_num(pin);
13     pwm_set_wrap(slice, 4095);
14     pwm_set_chan_level(slice, pwm_gpio_to_channel(pin), 0);
15     pwm_set_enabled(slice, true);
16 }
17
18 int main() {
19     stdio_init_all();
20     adc_init();
21
22     // Configurar ADC en pines GP26, GP27, GP28
23     adc_gpio_init(26); // Potenci metro Rojo
24     adc_gpio_init(27); // Potenci metro Azul (se intercambi con el
25     verde)
26     adc_gpio_init(28); // Potenci metro Verde (se intercambi con el
27     azul)
28
29     // Configurar PWM en pines del LED RGB
30     init_pwm(PIN_ROJO);
31     init_pwm(PIN_VERDE);
32     init_pwm(PIN_AZUL);
33
34     while (1) {
35         // Leer valores del ADC en el orden correcto
36         adc_select_input(0);
37         uint16_t valor_rojo = adc_read();
38         adc_select_input(2); // Se intercambiaron Verde y Azul
39         uint16_t valor_verde = adc_read();
40         adc_select_input(1);
41         uint16_t valor_azul = adc_read();
42
43         // Convertir a porcentaje
44         float porcentaje_rojo = (valor_rojo / 4095.0) * 100.0;
45         float porcentaje_verde = (valor_verde / 4095.0) * 100.0;
46         float porcentaje_azul = (valor_azul / 4095.0) * 100.0;
47
48         // Mostrar valores en el monitor serial
49         printf("Rojo: %.2f%% Verde: %.2f%% Azul: %.2f%%\n",
```

```

48     porcentaje_rojo, porcentaje_verde, porcentaje_azul);
49     // Invertir PWM solo para LED de nodo con n
50     uint16_t pwm_rojo = 4095 - valor_rojo;
51     uint16_t pwm_verde = 4095 - valor_verde;
52     uint16_t pwm_azul = 4095 - valor_azul;
53
54     // Aplicar PWM a los LEDs
55     pwm_set_gpio_level(PIN_ROJO, pwm_rojo);
56     pwm_set_gpio_level(PIN_VERDE, pwm_verde);
57     pwm_set_gpio_level(PIN_AZUL, pwm_azul);
58
59     // Espera antes de la siguiente lectura
60     sleep_ms(100);
61 }
62 }

```

Listing 1: RGB

creacion del CMakeLists

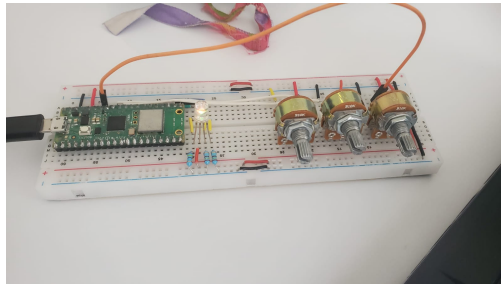
```

1 cmake_minimum_required(VERSION 3.13)
2
3 # Nombre del proyecto
4 project(Practica_13)
5
6 # Habilitar la Raspberry Pi Pico SDK
7 include(pico_sdk_import.cmake)
8 pico_sdk_init()
9
10 # Agregar el archivo fuente
11 add_executable(LED_RGB
12     LED_RGB.c
13 )
14
15 # Agregar las bibliotecas estandar de la Pico
16 target_link_libraries(LED_RGB
17     pico_stdlib
18     hardware_adc
19     hardware_pwm
20 )
21
22 # Habilitar la salida USB para el puerto serie
23 pico_enable_stdio_usb(LED_RGB 1)
24 pico_enable_stdio_uart(LED_RGB 0)
25
26 # Crear el archivo binario UF2
27 pico_add_extra_outputs(LED_RGB)

```

Listing 2: RGB

Armado



Explicación del código

El programa implementa el control de un LED RGB mediante señales PWM, utilizando tres potenciómetros como entrada analógica conectados al ADC del microcontrolador Raspberry Pi Pico. A continuación se describe el funcionamiento del código:

Librerías incluidas

- `#include <stdio.h>`: Permite el uso de funciones estándar de entrada/salida, como `printf()`.
- `#include "pico/stdlib.h"`: Habilita las funciones básicas del entorno de desarrollo de la Raspberry Pi Pico.
- `#include "hardware/adc.h"`: Habilita el uso del convertidor analógico-digital (ADC).
- `#include "hardware/pwm.h"`: Habilita el uso de las funciones relacionadas con modulación por ancho de pulso (PWM).

Definición de pines

Se definen tres macros para los pines GPIO que controlarán los canales del LED RGB:

```
#define PIN_ROJO 16
#define PIN_VERDE 18
#define PIN_AZUL 17
```

Cabe mencionar que los pines para los colores verde y azul fueron intercambiados respecto al orden tradicional.

Inicialización del PWM

La función `init_pwm(uint pin)` configura un pin como salida PWM:

- Asocia la función PWM al pin.
- Obtiene el número de *slice* correspondiente.
- Establece un valor de envolvente (wrap) de 4095, correspondiente a una resolución de 12 bits.
- Inicializa el canal con un nivel de señal 0.
- Habilita el canal PWM.

Función principal

La función `main()` ejecuta las siguientes acciones:

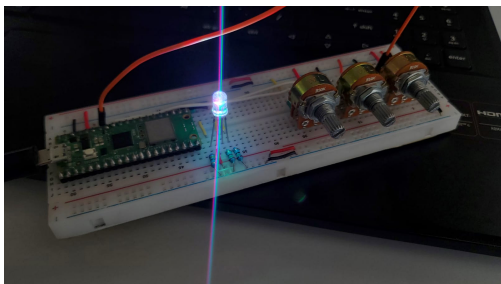
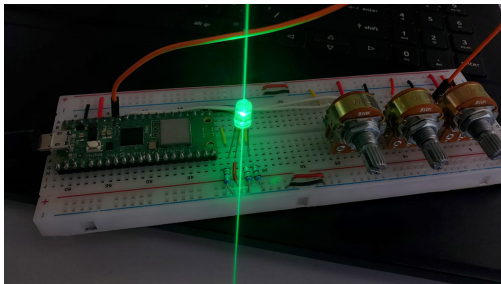
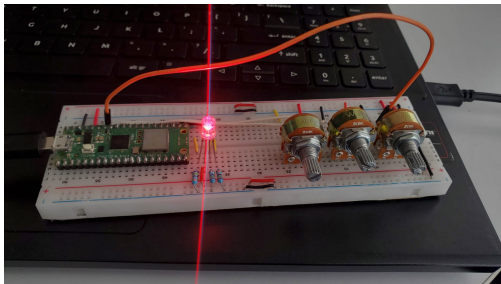
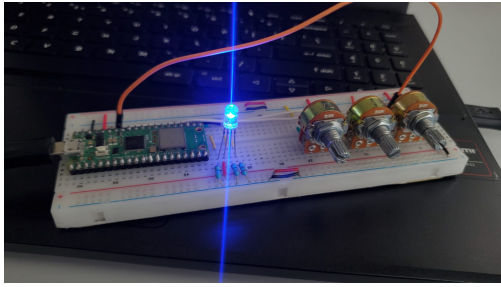
1. Se inicializa la comunicación por puerto serial con `stdio_init_all()` y el ADC con `adc_init()`.
2. Se configuran los pines GPIO 26, 27 y 28 como entradas analógicas para los potenciómetros.
3. Se inicializan los tres pines PWM correspondientes al LED RGB.

Bucle principal

En el ciclo `while (1)`, se realiza lo siguiente continuamente:

1. Se seleccionan las entradas del ADC para cada canal:
 - Entrada 0 (GPIO 26) para el canal rojo.
 - Entrada 2 (GPIO 28) para el canal verde.
 - Entrada 1 (GPIO 27) para el canal azul.
2. Se leen los valores analógicos, que se convierten en porcentajes para mostrar en consola usando `printf()`.
3. Debido a que el LED RGB es de ánodo común, se invierte el valor de PWM: a mayor entrada analógica, menor señal PWM.
4. Se actualizan los niveles PWM con `pwm_set_gpio_level()` para cada pin del LED RGB.
5. Finalmente, se espera 100 ms antes de repetir el ciclo, estabilizando la lectura y evitando saturación del monitor serial.

pruebas



Conclusion

La práctica permitió comprender de forma práctica el funcionamiento de la modulación por ancho de pulso (PWM) aplicada al control de intensidad luminosa en un LED RGB. Mediante el uso de potenciómetros como entradas analógicas, se logró mapear de manera directa los valores del ADC a señales PWM, generando así una gran variedad de colores en función de la posición de cada potenciómetro.

Además, se reforzó la importancia de considerar el tipo de LED (ánodo común o cátodo común), ya que esto afecta directamente la lógica de control de la señal PWM. La inversión del ciclo de trabajo en este caso fue esencial para obtener el comportamiento esperado.

Finalmente, esta práctica no solo permitió aplicar conocimientos de programación y electrónica, sino también fomentar la observación y el análisis en tiempo real del comportamiento de un sistema de control analógico-digital. Como posible mejora futura, se podría incorporar una interfaz gráfica que permita modificar los niveles RGB desde una aplicación o implementar un sistema de transición de colores automático.